sesa lab
SOFTWARE ENGINEERING
SALERNO

sesalab@unisa.it
sesa_lab
https://sesalabunisa.github.io/

Sustainable Software Engineering - Human Factors and Socio-Technical Sustainability in SE



Gemma Catolino | 2026-2027

Sustainable Software Engineering: The Human Core of Sustainable Software

Software sustainability is often described in terms of energy efficiency, maintainability, or cost. But a system cannot be truly sustainable if the people who design, develop, and maintain it are exhausted, disengaged, or treated unfairly.

Software development is not just a technical activity; it is a **social process** built on communication, trust, and shared knowledge.

Code, tools, and infrastructure are only one half of the equation — the other half is made of **human relationships and decisions**.

When teams fail to care for the human side — for motivation, learning, and well-being — the software becomes fragile, no matter how elegant the architecture looks.

Sustainable software, therefore, requires **sustainable people** who can keep contributing and improving without burning out.

Sustainable Software Engineering: From Algorithmic Fairness to Human

In the previous module we explored **fairness** in algorithms: are software systems equitable toward users? Now we turn inward and ask: are our **development processes** equitable toward the people who create those systems?



Sustainable Software Engineering: From Algorithmic Fairness to Human

In the previous module we explored **fairness** in algorithms: are software systems equitable toward users? Now we turn inward and ask: are our **development processes** equitable toward the people who create those systems?

An unfair process — one that rewards overwork, ignores some voices, or distributes recognition unevenly — slowly drains human energy.
Unfairness is a form of entropy: it creates hidden tension, erodes morale, and drives knowledge away. Just as algorithmic unfairness leads to social harm, **organizational unfairness** leads to unsustainable teams.

A fair environment, where work and recognition are balanced, is the ethical foundation on which sustainable engineering is built. (Greenberg 1990; Edmondson 1999)

Sustainable Software Engineering: Socio-Technical Systems: People and Architecture in Balance

Software engineering is a **social activity performed through technical artefacts**.
Every commit, review, or merge is both a technical change and a communicative act — it transfers knowledge, negotiates meaning, and coordinates responsibility.

A software system therefore embodies **two intertwined networks**:

1. The **technical network** — modules, dependencies, build pipelines, architectures.
2. The **social network** — developers, teams, communication channels, and review interactions.

These networks co-evolve.
When the architecture changes, collaboration patterns must adapt; when the team reorganizes, communication must match technical dependencies.
Sustainability emerges when the *two remain aligned*.

When architecture evolves but team communication does not, misalignment appears. Cataldo et al. (2008) called this **socio-technical congruence** — the degree of alignment between coordination needs and actual communication.

Socio-technical congruence is the **degree of alignment between coordination needs implied by the technical structure and the actual coordination that occurs among developers**.

In simpler terms:

Do the people who need to talk actually talk?

A team has **high STC** when developers who work on interdependent modules communicate frequently and effectively.
It has **low STC** when code is tightly coupled but the people responsible are isolated — creating invisible coordination gaps.

Sustainable Software Engineering: How Socio-Technical Congruence Is Measured

$$\text{STC} = \frac{\text{Number of coordination needs that are actually communicated}}{\text{Total number of coordination needs}}$$

If 10 pairs of developers **need** to coordinate (because their modules depend on each other) and 7 of those pairs **actually communicate**, then:

STC = 7 /10 = 0.7

→ **70 % congruence** means most coordination happens as needed — good alignment.

Sustainable Software Engineering: How Socio-Technical Congruence Is Measured

Low STC is a hidden sustainability risk:

- It increases misunderstandings and rework.
- It isolates maintainers and concentrates cognitive load.
- It fuels *social debt* and *human fatigue*.

High STC, instead, reflects a **healthy socio-technical ecosystem**:

- coordination flows match technical complexity,
- work is predictable and recoverable,
- knowledge is distributed instead of trapped.

Maintaining socio-technical congruence therefore means **aligning the social system of collaboration with the technical system of dependencies** — a necessary condition for sustainable development.

This alignment can be supported through:

- transparent ownership models,
- open communication across teams,
- architectural visibility tools (dependency maps, review analytics).

Sustainable Software Engineering: From Technical Debt to Social and Human Debt

In software engineering, the concept of **debt** helps describe how short-term decisions can create long-term costs. Originally, *technical debt* referred to the hidden cost of quick fixes in code — solutions that save time now but demand heavy rework later.
Over time, researchers have extended this metaphor to the **social** and **human** dimensions of software development.

- **Technical Debt** arises from shortcuts in design, testing, or architecture.
  It affects maintainability and code quality.
- **Social Debt** arises when **collaboration and communication structures deteriorate** — for example, when teams avoid coordination, skip documentation, or centralize knowledge in a few individuals.
  Tamburri et al. (2015) define it as the "sum of social and organizational shortcuts that reduce long-term team effectiveness."
- **Human Debt** arises when **individual well-being and learning** are neglected — when teams accumulate stress, fatigue, or resentment instead of renewing their energy and motivation. As Blomström (2021) explains, it includes every missed opportunity to make work more humane and sustainable.

Sustainable Software Engineering: From Technical Debt to Social and Human Debt

These three forms of debt are **deeply interconnected**:
technical shortcuts often require social shortcuts (less review, less discussion), which then create human strain (overwork, frustration).

Likewise, human exhaustion leads to new social gaps and more technical errors — a self-reinforcing cycle of unsustainability.

Managing debt sustainably therefore means addressing *all three* dimensions together: cleaning the code, maintaining healthy collaboration, and protecting people's energy.

We will explore *social debt* more deeply through the lens of **community smells** — recurring patterns of unhealthy collaboration that threaten both software quality and team well-being.

Sustainable Software Engineering: Why Human Factors Matter

Research consistently shows that **happy, motivated developers build better software**.

- Graziotin et al. (2014) demonstrated that developers in a positive emotional state solve problems faster and make fewer logical errors.
- Edmondson (1999) showed that teams with psychological safety are more willing to report mistakes — and therefore fix them earlier.
- Lenberg et al. (2015) found that unclear roles and poor communication directly reduce industrial productivity.

These studies reveal that human and organizational variables are *not secondary*; they are measurable determinants of software quality and sustainability.



Sustainable Software Engineering: Motivation as Renewable Energy

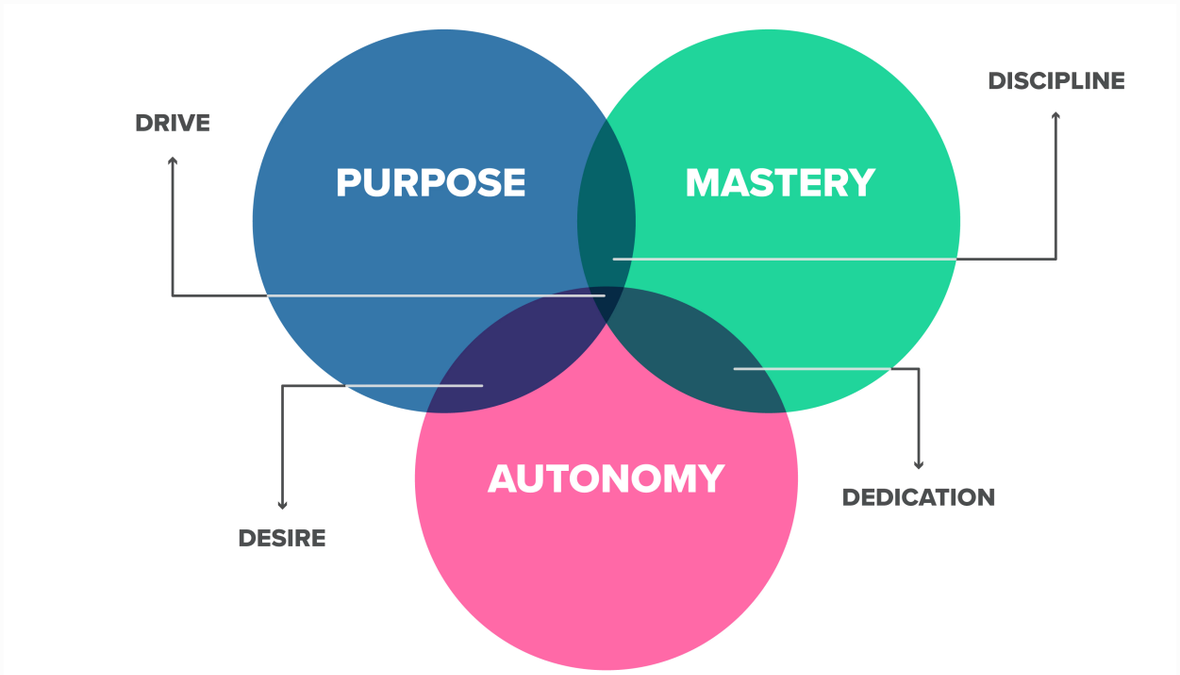
Self-Determination Theory (Deci & Ryan 2000) identifies three core human needs that sustain motivation:

- **Autonomy:** the ability to make meaningful decisions about one's work.
- **Mastery:** the opportunity to improve and feel competent.
- **Purpose:** the sense that work contributes to something valuable.

When organizations support these needs, motivation becomes a *renewable resource*.

When they suppress them — with rigid control, meaningless tasks, or lack of recognition — motivation is consumed like fuel.

In the long run, sustainability depends on **how well we renew human motivation**, not how fast we burn it.



Sustainable Software Engineering: Burnout - The Unsustainable State

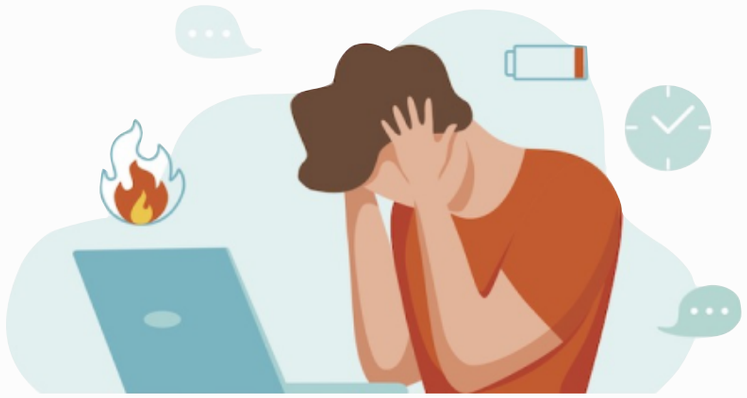
The World Health Organization defines **burnout** as exhaustion resulting from chronic workplace stress that has not been successfully managed.

In software engineering, burnout is often invisible until it is too late:
developers work nights to meet deadlines, respond to endless alerts, or face constant context switching.
Graziotin et al. (2018) showed that unhappy developers communicate less and withdraw from collaboration.

Fagerholm et al. (2015) observed that poor onboarding and mentoring accelerate burnout among newcomers.

Burnout weakens both individuals and the project's memory.

Preventing it is not optional — it is a design requirement for sustainable systems.



Sustainable Software Engineering: Team Morale and Collective Resilience

Team morale represents the emotional temperature of a team — how confident, optimistic, or exhausted people feel.

Morale influences creativity, willingness to take responsibility, and resistance to stress.

Graziotin et al. (2018) found that high morale encourages participation in code reviews and feedback loops.

Low morale leads to silence and disengagement, which quietly destroy sustainability.

Healthy morale arises from inclusion, recognition, and psychological safety — not from pressure or competition.

It is cheaper to maintain morale than to repair the damage when it collapses.



Sustainable Software Engineering: Psychological Safety - Learning Without Fear

Amy Edmondson (1999) defined **psychological safety** as the shared belief that "the team is safe for interpersonal risk-taking." In psychologically safe teams, people can admit mistakes, ask questions, or propose changes without fear of ridicule or punishment. This climate enables *collective learning*: problems surface early, feedback becomes normal, and innovation accelerates. In sustainability terms, psychological safety is like an immune system: it detects and corrects small failures before they become chronic disease.



Sustainable Software Engineering: Psychological Safety: Google Aristotle Project

Launched in **2012,** over a period of two years, the project collected data by studying **180** Google teams—115 engineering teams and 65 sales teams. Researchers examined over 250 different team attributes, collecting a "mountain of data" through quantitative surveys, interviews, and existing performance reviews. The project ran until roughly **2014**, when the findings began to be published internally and externally.

They found that the most important factor in team effectiveness is **not** who is on the team (personality, background, demographics), but **how** the team members interact and structure their work. The single most important dynamic identified was **psychological safety**, where team members feel safe to take risks and admit mistakes without fear of judgment. The five key dynamics identified for successful teams were:

1. **Psychological Safety:** Feeling safe to speak up, ask questions, and make mistakes.
2. **Dependability:** Being able to count on each other to get work done well and on time.
3. **Structure & Clarity:** Clear roles, plans, and goals.
4. **Meaning of Work:** Finding a personal sense of purpose in the work.
5. **Impact of Work:** Believing the work makes a positive difference.

Sustainable Software Engineering: Fairness Inside Teams

While psychological safety allows individuals to vocalize ideas and concerns without fear, **organizational justice** determines whether those contributions are heard, valued, and addressed equitably. Justice essentially provides the trustworthy infrastructure that stabilizes safety over the long term.

Drawing on Greenberg (1990), organizational justice describes the perception of fairness in the workplace across three interconnected dimensions:

1. **Distributive Justice**: The balance between effort and reward. Recognition, workload, and opportunities should align with actual contributions. Discrepancies here lead to cynicism and "human debt."
2. **Procedural Justice**: Transparent and consistent processes. Decisions regarding promotions, code ownership, or reviews should follow clear, predictable criteria, reducing anxiety and strengthening trust.
3. **Interactional Justice**: Respect and dignity in daily communication. Even when outcomes are unfavorable, honest and respectful explanations maintain morale and psychological safety.

Sustainable Software Engineering: Measuring Human Sustainability

If we want to treat sustainability as a serious engineering goal, we must be able to observe and measure it — not only in code, but also in the *human and social fabric* of development.

Quantitative indicators do not replace empathy, but they help us detect early warning signs of stress, isolation, or fragility within teams.

The following metrics capture some of the *measurable footprints of human sustainability*:

- Bus Truck Factor
- Developer Churn
- Knowledge Concentration and Ownership
- Communication and Review Networks

Sustainable Software Engineering: Bus (Truck) Factor

Definition: the minimum number of developers whose loss would bring a project to a standstill.

If only one or two people understand key parts of the system, the team is living dangerously.

- A *low bus factor* indicates high **knowledge concentration** and high risk: the departure of one person may cause major disruption.
- A *high bus factor* reflects knowledge sharing, redundancy, and mentoring — qualities that sustain the system's memory.

How to measure:

You can estimate it by analyzing code ownership data (e.g., via git blame, commit frequency, or review logs).

Studies show that higher bus factors correlate with more maintainable and resilient projects (Rigby & Storey, 2016; Meneely & Williams, 2011).

Interpretation:

The bus factor is not just a number — it is an indicator of **organizational health**.

A low bus factor often signals *social debt* and potential burnout risk for key individuals.

Sustainable Software Engineering: Developer Churn

Definition: the rate at which contributors join or leave a software project over time.

It reflects the *stability* of the team that maintains the codebase.

- **High churn** means many developers are joining and leaving quickly.
  This causes repeated onboarding, loss of context, and wasted effort.
- **Low churn** means people stay longer, and knowledge remains inside the team.
  However, zero churn for long periods can also signal stagnation or lack of openness.

How to measure:

Count how many developers start contributing (inflow) and how many stop contributing (outflow) during a defined period — for example, every six months.

Version-control history (e.g., GitHub logs) or issue-tracker data can show these patterns clearly.

Interpretation:

Churn is an early signal of *human and social sustainability*.

A sudden rise in outflow may indicate burnout, unfair workload, or lack of psychological safety. Tracking churn helps teams notice when their culture starts to push people away.

Sustainable Software Engineering: Knowledge Concentration and Ownership

Definition: the extent to which a small subset of developers dominate contributions or decision-making.

Why it matters:

When knowledge is concentrated in a few people:

- collaboration becomes slower;
- the system depends on informal heroes;
- review diversity decreases;
- others have little room to learn.

Measurement approaches:

Network analysis can quantify ownership distribution using developer–file graphs (Meneely & Williams, 2011).

Tools such as *PyDriller (indirectly)* or *CodeScene* help compute ownership entropy — lower entropy means higher concentration.

Interpretation:

High knowledge concentration = **socio-technical fragility**.

It also creates **fairness issues**, because influence and recognition become unevenly distributed.

Sustainable Software Engineering: Communication and Review Networks

Beyond code, we can measure **who talks to whom**.

Social network analysis of issue discussions or code reviews reveals participation patterns:

- Dense, balanced networks → healthy collaboration.
- Sparse or polarized networks → risk of social fragmentation (a community smell we'll study in Lecture 3).

By overlaying these social graphs with architectural dependencies (Cataldo et al., 2008), we can assess **socio-technical congruence** — another proxy for sustainability.



Numbers alone are not enough!!!!

Qualitative evidence such as developer surveys, sentiment analysis of commit messages, or retrospective notes help capture *how people feel*.

Research by Graziotin et al. (2018) shows that self-reported happiness correlates strongly with participation and productivity.

Combining quantitative and qualitative data provides a more complete picture of human sustainability.

Sustainable Software Engineering: Unsustainable Cultural Patterns

Certain recurring habits make teams unsustainable even when they appear productive:

- **Hero culture:** a few individuals are always expected to save the project.
- **Perpetual urgency:** constant “crunch mode” leaves no room for learning.
- **Toxic silence:** people avoid raising issues for fear of blame.
- **Neglected onboarding:** new members struggle and experienced ones burn out.

These patterns accumulate human and socio-technical debt.

They produce short-term gains but long-term fragility.



Sustainable Software Engineering: Sustainable Practices and Cultural Repair

Sustainable organizations work differently.

They embed *reflection and recovery* into daily routines:

- Allocate time for learning and documentation.
- Rotate demanding roles like on-call duties.
- Conduct *blameless post-mortems* to transform failure into learning.
- Value steady, balanced contribution over heroics.

Companies such as Etsy and Google's teams have shown that these practices reduce stress and improve reliability.

Sustainability, in this sense, is not luck — it is a **systemic property built through design**.



Sustainable Software Engineering: The Interdependence of Sustainability Dimensions

Human (Individual), technical, economic, and environmental sustainability are intertwined:

Healthy teams → cleaner code → less waste → lower energy and cost.

Stressed teams → quick fixes → rework → wasted computation and human energy.

Focusing on one dimension while neglecting another leads to imbalance.

True sustainability emerges only when we care for *both* systems: the technical one that runs on electricity and the human one that runs on empathy.



Sustainable Software Engineering: Fairness and Sustainability: Two Faces of Ethics

Fairness protects people **now**; sustainability protects them **over time**. An unfair system may succeed briefly but eventually collapses under its own inequity. Sustainability gives fairness a timescale — it turns justice into longevity. A sustainable engineering culture is fair, transparent, inclusive, and humane. It does not trade short-term performance for long-term well-being.



Sustainable Software Engineering: Key Insights to Remember

- Sustainability in software is **socio-technical**: people and code evolve together.

- **Human debt** is real: neglecting well-being and fairness erodes capacity like technical debt erodes code quality.

- Motivation, morale, and safety are *measurable design factors*.

- Sustainable teams protect their energy through equity, psychological safety, and balance.

- Fairness and sustainability form the ethical and temporal foundations of modern software engineering.

Sustainable Software Engineering: References

1. Blomström, D. (2021, Aug 6). *How to Recognise and Reduce HumanDebt*. InfoQ. Retrieved from https://www.infoq.com/articles/human-debt/
2. Cataldo, Marcelo, James D. Herbsleb, and Kathleen M. Carley. "Socio-technical congruence: a framework for assessing the impact of technical and work dependencies on software development productivity." *Proceedings of the Second ACM-IEEE international symposium on Empirical software engineering and measurement*. 2008.
3. Deci, Edward L., and Richard M. Ryan. "The" what" and" why" of goal pursuits: Human needs and the self-determination of behavior." *Psychological inquiry* 11.4 (2000): 227-268.
4. Edmondson, Amy. "Psychological safety and learning behavior in work teams." *Administrative science quarterly* 44.2 (1999): 350-383.
5. Forsgren, Nicole, Jez Humble, and Gene Kim. *Accelerate: The science of lean software and devops: Building and scaling high performing technology organizations*. IT Revolution, 2018.
6. Graziotin, Daniel, Xiaofeng Wang, and Pekka Abrahamsson. "Happy software developers solve problems better: psychological measurements in empirical software engineering." *PeerJ* 2 (2014): e289.
7. Graziotin, Daniel, Fabian Fagerholm, Xiaofeng Wang, and Pekka Abrahamsson. "What happens when software developers are (un) happy." *Journal of Systems and Software* 140 (2018): 32-47.
8. Greenberg, Jerald. "Organizational justice: Yesterday, today, and tomorrow." *Journal of management* 16.2 (1990): 399-432.
9. Lenberg, Per, Robert Feldt, and Lars Göran Wallgren. "Human factors related challenges in software engineering--an industrial perspective." *2015 ieee/acm 8th international workshop on cooperative and human aspects of software engineering*. IEEE, 2015
10. Meneely, Andrew, and Laurie Williams. "Socio-technical developer networks: Should we trust our measurements?." *Proceedings of the 33rd international conference on software engineering*. 2011.
11. Tamburri, Damian A., Philippe Kruchten, Patricia Lago, and Hans van Vliet. "Social debt in software engineering: insights from industry." *Journal of Internet Services and Applications* 6, no. 1 (2015): 10.
12. Tamburri, Damian A. "Software architecture social debt: Managing the incommunicability factor." *IEEE Transactions on Computational Social Systems* 6.1 (2019): 20-37.

sesa lab
SOFTWARE ENGINEERING
SALERNO

sesalab@unisa.it
sesa_lab
https://sesalabunisa.github.io/

Sustainable Software Engineering - Human Factors and Socio-Technical Sustainability in SE



Gemma Catolino | 2026-2027